

Documentação Técnica

Case Integração — Sistema de Pedidos → CRM

Projeto: CaseIntegracao · Stack: C# / ASP.NET Core (.NET 9)

Estrutura: Api + Core + Tests · Persistência: arquivo JSON

Documento complementar ao README (visão aprofundada)

1. Objetivo e contexto

O serviço atua como **integrador** entre dois sistemas:

- **Sistema de Pedidos** — dispara webhooks quando um pedido é criado, atualizado ou cancelado.
- **CRM** — mantém clientes e pedidos via API REST (neste projeto, um mock embutido no mesmo host).

O integrador recebe o evento, garante o cliente no CRM, sincroniza o pedido e trata problemas típicos de integração: **eventos fora de ordem**, **falhas temporárias** na API de destino e **reenvios duplicados**.

2. Escopo funcional

1. Endpoint HTTP para receber o webhook do sistema de pedidos.
2. Se o cliente não existir no CRM, criá-lo antes de sincronizar o pedido.
3. Criar ou atualizar o pedido no CRM conforme o tipo de evento.
4. Idempotência por `eventId`.
5. Ordenação por `occurredAt` (estado final = mais recente).
6. Retry sensato em falhas 5xx, 429 e timeout, sem perder o evento.

Diferenciais mantidos: exponential backoff, circuit breaker (Polly), métricas de sincronização e reflexão sobre escala com fila.

3. Arquitetura da solução

3.1 Estrutura de projetos (Api + Core)

Projeto	Papel	Conteúdo principal
CaseIntegracao.Api	Entrada HTTP, Swagger, mock CRM, worker	Controllers, <code>RetryBackgroundService</code> , <code>Program.cs</code>
CaseIntegracao.Core	Um csproj com camadas internas	<code>Domain/</code> , <code>Application/</code> , <code>Infrastructure/</code> , <code>DI</code>
CaseIntegracao.Tests	Testes automatizados	Idempotência, ordenação, cancelamento, retry

Dois projetos na solution; dentro do Core as pastas deixam explícito o que é domínio, aplicação e infraestrutura (Clean Architecture leve, sem vários .csproj).

3.2 Pastas internas do Core

Pasta	Responsabilidade
Domain/Entities	EventoIntegracao , ProjecaoPedido , DadosCliente
Domain/Enums	Tipos de evento, status de pedido e de processamento
Domain/Interfaces	Portas: repositories, ICrmClient , IMetricasCollector
Domain/Services	PoliticaOrdenacaoEventos
Application/Services	EventoPedidoProcessor , ConsultaEventosService
Application/DTOs , Mapping , Options	Contrato webhook, mapeamento e opções de retry
Infrastructure/Persistence	ArquivoJsonStore e repositórios em arquivo
Infrastructure/Crm	CrmHttpClient , DTOs e store do mock
Infrastructure/Metrics	Coletor em memória

3.3 Fluxo ponta a ponta

```
Sistema de Pedidos
|
▼
POST /api/webhooks/orders (WebhooksController)
|
▼
EventoPedidoProcessor.ReceberAsync (Core)
├─ Idempotência (eventId já sincronizado?)
├─ Persiste EventoIntegracao (data/events.json)
├─ PoliticaOrdenacaoEventos (occurredAt)
├─ ICrmClient.GarantirClienteAsync
├─ ICrmClient.UpsertPedidoAsync
├─ Atualiza ProjecaoPedido (data/orders.json)
├─ sucesso → Status = Sincronizado
├─ falha transitória → Status = Falhou + NextRetryAt
|
▼
RetryBackgroundService (Api)
|
▼
reprocessa / CartaMortua após MaxAttempts
```

4. Modelagem de estado interno

4.1 EventoIntegracao

Representa um evento recebido e seu ciclo de vida no pipeline de sincronização.

Campo	Descrição
EventId	Identificador único do evento (chave de idempotência)
EventType	PedidoCriado , PedidoAtualizado , PedidoCancelado
OccurredAt	Instante de negócio usado na ordenação
OrderId, status, valor, moeda, cliente	Payload do pedido
Status	Status do <i>processamento</i> (não confundir com status do pedido)
AttemptCount / NextRetryAt / LastError	Controle de retry

StatusProcessamentoEvento

Status	Significado
Recebido	Evento persistido, ainda não processado com sucesso
Processando	Em processamento na tentativa atual
Sincronizado	Aplicado na projeção e no CRM
IgnoradoObsoleto	Evento mais antigo que o estado já conhecido
Falhou	Falha transitória; aguarda retry
CartaMortua	Esgotou tentativas; evento preservado para auditoria

4.2 ProjecaoPedido

Estado local consolidado do pedido. Guarda LastOccurredAt para decidir se um novo evento deve ser aplicado. Fonte de GET /api/orders/{orderId} .

StatusPedido	Origem típica
Pendente	order.created / status pending
Confirmado	order.updated / status confirmed
Cancelado	order.canceled (forçado pelo tipo de evento)

4.3 Persistência

- `data/events.json` — eventos e status de processamento
- `data/orders.json` — projeções de pedido
- `data/crm/customers.json` e `orders.json` — store do CRM mock

Escrita via `ArquivoJsonStore<T> : SemaphoreSlim` + replace atômico. Justificativa: simplicidade e aderência ao case, sem Docker/DB.

5. Lógica de negócio detalhada

5.1 Recepção do webhook

1. `WebhooksController` recebe o JSON; se `eventId` estiver vazio, usa `X-Idempotency-Key`.
2. `WebhookMapper` converte o contrato inglês em entidades de domínio.
3. Em `order.canceled`, o status vira `Cancelado` automaticamente.
4. `EventoPedidoProcessor.ReceberAsync` executa o pipeline.

5.2 Idempotência

Se já existe evento com o mesmo `eventId` em `Sincronizado` ou `IgnoradoObsoleto`, a API responde `alreadyProcessed: true` e **não** chama o CRM novamente. Cada evento de negócio novo (`create` → `update` → `cancel`) precisa de um `eventId` distinto.

5.3 Ordenação de eventos

`PoliticaOrdenacaoEventos` compara `occurredAt` com `ProjecaoPedido.LastOccurredAt`: evento mais antigo → `IgnoradoObsoleto`; mais novo ou igual → aplica no CRM e na projeção.

5.4 Sincronização com o CRM

1. **GarantirClienteAsync** — GET; se 404, POST; se existe, PUT.
2. **UpsertPedidoAsync** — GET; se 404, POST; senão PUT. Status enviado em inglês (`pending` / `confirmed` / `canceled`).

5.5 Tratamento de falhas e retry

Polly (HttpClient): retries curtos para 429/5xx + circuit breaker (5 falhas → abre 15s).

Aplicação: falha transitória → `Falhou` + backoff exponencial com jitter; `RetryBackgroundService` reprocessa; após `MaxAttempts` → `CartaMortua`.

CRM mock falha ~15% (`CrmMock.FailureRate`): 429, 500 ou timeout real (delay padrão 12s > timeout do client 10s).

6. Catálogo de endpoints

6.1 API de integração

Método	Rota	Descrição	Respostas
POST	/api/webhooks/orders	Recebe evento do sistema de pedidos	200 (já processado / sucesso), 202 (aceito / retry), 400
GET	/api/events	Lista eventos e status	200
GET	/api/events/{eventId}	Detalhe do evento	200 / 404
GET	/api/orders/{orderId}	Projeção local do pedido	200 / 404
GET	/api/metrics	Contadores de sync / falha / retry / idempotência	200

6.2 Header opcional

Header	Uso
X-Idempotency-Key	Usado como eventId somente se o body vier sem eventId.

6.3 Contrato do webhook

```
{
  "eventId": "evt_01HXYZ",
  "eventType": "order.created | order.updated | order.canceled",
  "occurredAt": "2026-07-27T18:00:00Z",
  "data": {
    "orderId": "ord_123",
    "status": "pending | confirmed | canceled",
    "totalAmount": 150.90,
    "currency": "BRL",
    "customer": {
      "externalId": "cust_456",
      "name": "Ana Silva",
      "email": "ana@email.com",
      "document": "12345678901"
    }
  }
}
```

Em `order.canceled`, `data.status` é opcional/ignorado — o domínio força cancelado.

6.4 CRM mock

Método	Rota	Descrição
GET	/crm/customers/{externalId}	Busca cliente
POST	/crm/customers	Cria cliente
PUT	/crm/customers/{externalId}	Atualiza cliente
GET	/crm/orders/{orderId}	Busca pedido
POST	/crm/orders	Cria pedido
PUT	/crm/orders/{orderId}	Atualiza/cancela pedido

7. Configuração

Chave	Uso
Storage:DataPath	Pasta dos JSON (data)
Crm:BaseUrl	Base do CRM (dev: mesmo host)
CrmMock:FailureRate	Taxa de falha do mock (~0.15)
CrmMock:TimeoutDelaySeconds	Delay do timeout simulado (padrão 12)
Retry:MaxAttempts	Tentativas antes de carta morta
Retry:BaseDelaySeconds / MaxDelaySeconds	Backoff
Retry:PollIntervalSeconds	Intervalo do worker

8. Observabilidade

- Logs estruturados (ILogger) em sucessos, falhas, retries e idempotência.
- Métricas em GET /api/metrics .
- Swagger em /swagger (inclui mock CRM).

9. Testes

- mesmo eventId duas vezes → um único efeito;
- evento antigo após um mais novo → estado = mais recente;
- falha transitória → retry e sync;
- order.canceled força status cancelado.

```
dotnet test
```

10. Como executar

```
cd CaseIntegracao
dotnet restore
dotnet build
dotnet test
dotnet run --project CaseIntegracao.Api
```

- API: `http://localhost:5080`
- Swagger: `http://localhost:5080/swagger`
- Postman: `postman/CaseIntegracao.postman_collection.json`
- .http: `CaseIntegracao.Api/CaseIntegracao.Api.http`

11. Decisões e trade-offs

Decisão	Por quê	Trade-off
Api + Core com pastas Domain/Application/Infrastructure	Case enxuto + camadas legíveis na Solution Explorer	Menos isolamento físico que vários .csproj
Arquivo JSON	Zero dependência externa	Concorrência limitada
Sem fila externa	Case não exige; sync + worker bastam	Menos desacoplamento sob carga
CRM mock no mesmo processo	Demo/Swagger/testes locais fáceis	Não isola rede real
Polly + retry de aplicação	Retries curtos na borda + DLQ de negócio	Duas camadas de resiliência
JSON do webhook em inglês; código em PT-BR	Contrato de integração + domínio legível	Dois idiomas no projeto

12. Escala com fila (reflexão)

Em produção, o webhook validaria o payload, gravaria um inbox e publicaria em fila (SQS, RabbitMQ ou Azure Service Bus). Workers competiriam com visibility timeout e DLQ. Idempotência e projeção ficariam em store compartilhado (banco), permitindo múltiplas instâncias.

13. O que mudaria com mais tempo

- Banco (ou outbox/inbox) em vez de arquivo
- Testes de integração com `WebApplicationFactory`
- Assinatura HMAC no webhook
- OpenTelemetry / tracing
- Exemplos OpenAPI por `eventType`

14. Mapa de pastas

```
CaseIntegracao/  
├─ CaseIntegracao.Api/      Controllers, Workers, Swagger, Program  
├─ CaseIntegracao.Core/  
│  ├─ Domain/              Entities, Enums, Interfaces, PoliticaOrdenacaoEventos  
│  ├─ Application/         Processor, DTOs, Mapping, Options  
│  ├─ Infrastructure/      Persistence, Crm, Metrics  
│  └─ DependencyInjection.cs  
├─ CaseIntegracao.Tests/    Testes xUnit  
├─ postman/                 Collection Postman  
├─ docs/                    Esta documentação (PDF)  
└─ README.md                Guia de entrega do case
```

Fim do documento técnico — Case Integração Pedidos → CRM (Api + Core).